

Vorlesung 6: Grundlagen des Prompt-Engineering



Inhalte

1. Was ist Prompt-Engineering?
2. Grundprinzipien: Klarheit, Kontext, Struktur
3. Prompting-Techniken (Zero-shot, Few-shot, Chain-of-Thought)
4. Role-based und Template-basiertes Prompting
5. Common Pitfalls und wie man sie vermeidet
6. Prompt-Optimierung und Iteration
7. Praxisbeispiele: REST-Server und Web-Anwendung
8. Praktische Übungen

Was ist Prompt-Engineering?

Definition: Prompt-Engineering

Prompt-Engineering ist die systematische Gestaltung und Optimierung von Eingaben für Large Language Models.

- **Prompt:** Eine Eingabe oder Anweisung an ein LLM
- **Engineering:** Systematische Gestaltung und Optimierung
- **Ziel 1:** Maximale Qualität und Relevanz der LLM-Ausgaben
- **Ziel 2:** Vermeiden von Fehlern wie Haluzinieren
- **Ansatz:** Iterative Verbesserung durch strukturierte Methoden
- <https://www.promptingguide.ai/>

Warum ist Prompt-Engineering wichtig?

Die Qualität der Prompts bestimmt maßgeblich die Qualität der Ergebnisse.

- **Qualität:** Bessere Prompts = bessere Ergebnisse
- **Effizienz:** Weniger Iterationen, schnellere Ergebnisse
- **Kosten:** Optimierte Prompts = weniger Token = niedrigere Kosten
- **Zuverlässigkeit:** Konsistente und vorhersagbare Ausgaben

Die Herausforderung

LLMs interpretieren Prompts unterschiedlich, was die systematische Gestaltung notwendig macht.

- LLMs interpretieren Prompts unterschiedlich
- Kleine Änderungen können große Auswirkungen haben
- Keine universelle "beste" Lösung
- Kontextabhängig und modellabhängig

Grundprinzipien: Klarheit, Kontext, Struktur

Prinzip 1: Klarheit

Klarheit bedeutet präzise, eindeutige und korrekte Formulierungen ohne Mehrdeutigkeiten.

- **Präzise Formulierung:** Eindeutige Anweisungen
- **Vermeidung von Mehrdeutigkeiten:** Klare Erwartungen setzen
- **Spezifität:** Je spezifischer, desto besser
- **Direktheit:** Direkte Fragen statt umständlicher Formulierungen
- **Korrektheit:** Verwende die richtigen Begriffe, vermeide Schreibfehler

Klarheit: Beispiele

Vage Formulierungen führen zu unklaren Ergebnissen, präzise Prompts zu besseren Ausgaben.

Schlecht:

Mach etwas mit Daten.

Besser:

Analysiere die CSV-Datei 'sales.csv' und erstelle eine Zusammenfassung der monatlichen Verkäufe nach Produktkategorie.

Prinzip 2: Kontext

Relevanter Kontext hilft dem LLM, die Aufgabe besser zu verstehen und passende Ergebnisse zu generieren.

- **Relevante Informationen:** Alle nötigen Details bereitstellen
- **Hintergrundwissen:** Domain-spezifischer Kontext
- **Zielsetzung:** Klare Angabe des gewünschten Ergebnisses
- **Rahmenbedingungen:** Einschränkungen und Anforderungen

Kontext: Beispiele

Ohne Kontext kann das LLM nicht wissen, was genau gewünscht ist.

Schlecht:

Schreibe Code für eine API.

Besser:

Erstelle eine REST-API mit Node.js und Express für ein Todo-Management-System. Die API soll CRUD-Operationen unterstützen, JSON verwenden und Fehlerbehandlung implementieren.

Prinzip 3: Struktur

Eine klare Struktur verbessert die Lesbarkeit und hilft dem LLM, die Anforderungen zu verstehen.

- **Organisation:** Logische Anordnung der Informationen
- **Formatierung:** Nutzung von Markdown, Listen, Abschnitten
- **Hierarchie:** Wichtige Informationen hervorheben
- **Lesbarkeit:** Klare visuelle Struktur

Struktur: Beispiele

Unstrukturierte Prompts sind schwer zu verstehen, strukturierte Prompts führen zu besseren Ergebnissen.

Schlecht:

Erstelle eine Funktion die Zahlen sortiert und dann das Maximum zurückgibt aber nur wenn die Liste nicht leer ist.

Besser:

Erstelle eine Python-Funktion mit folgenden Anforderungen:

- Eingabe: Liste von Zahlen
- Verarbeitung: Sortiere die Liste aufsteigend
- Ausgabe: Maximum der sortierten Liste
- Fehlerbehandlung: Rückgabe None, wenn Liste leer ist

Prompting-Techniken

Zero-shot Prompting

Zero-shot Prompting verwendet einen einzelnen Prompt ohne Beispiele und eignet sich für einfache Aufgaben.

- **Definition:** Ein einzelner Prompt ohne Beispiele
- **Anwendung:** Für einfache, gut verstandene Aufgaben
- **Vorteile:** Schnell, wenig Token-Verbrauch
- **Nachteile:** Kann bei komplexen Aufgaben unzureichend sein

Zero-shot: Beispiel

Zero-shot Prompts sind direkt und erfordern keine Vorbeispiele.

Klassifiziere folgenden Text als positiv, neutral oder negativ:

"Das Produkt funktioniert einwandfrei und hat meine Erwartungen übertroffen."

Few-shot Prompting

Few-shot Prompting demonstriert das gewünschte Pattern durch mehrere Beispiele.

- **Definition:** Prompt mit mehreren Beispielen
- **Anwendung:** Für komplexere Aufgaben, Pattern-Learning
- **Vorteile:** Bessere Ergebnisse durch Demonstration
- **Nachteile:** Höherer Token-Verbrauch

Few-shot: Beispiel

Beispiele zeigen dem LLM das gewünschte Format und Verhalten.

Übersetze folgende Sätze ins Englische:

Deutsch: "Guten Morgen"

Englisch: "Good morning"

Deutsch: "Wie geht es dir?"

Englisch: "How are you?"

Deutsch: "Ich lerne Deutsch"

Englisch: ?

- <https://www.promptingguide.ai/techniques/fewshot>

Chain-of-Thought (CoT)

Chain-of-Thought regt das LLM an, Probleme Schritt für Schritt zu lösen.

- **Definition:** Schritt-für-Schritt-Denken anregen
- **Anwendung:** Für komplexe logische Probleme
- **Vorteile:** Bessere Reasoning-Fähigkeiten
- **Technik:** "Lass uns Schritt für Schritt vorgehen"
- **Logik-Fragen:** Bei logischen oder mathematischen Fragen unverzichtbar

Chain-of-Thought: Beispiel

CoT führt zu besseren Ergebnissen bei komplexen logischen Problemen.

Löse folgendes Problem Schritt für Schritt:

Ein Geschäft verkauft 120 Äpfel. Am Morgen werden 45 verkauft, am Nachmittag 30. Wie viele Äpfel bleiben übrig?

Lass uns Schritt für Schritt vorgehen:

1. Start: 120 Äpfel
 2. Nach Morgen: $120 - 45 = 75$ Äpfel
 3. Nach Nachmittag: $75 - 30 = 45$ Äpfel
- Antwort: 45 Äpfel bleiben übrig.

- <https://www.promptingguide.ai/techniques/cot>

Role-based Prompting

Role-based Prompting weist dem LLM eine spezifische Rolle zu, um domänenspezifische Expertise zu nutzen.

- **Definition:** LLM eine spezifische Rolle zuweisen
- **Anwendung:** Für domänenspezifische Aufgaben
- **Vorteile:** Bessere Kontextualisierung, konsistentere Ausgaben
- **Beispiele:** "Du bist ein Senior-Softwareentwickler..."

Role-based: Beispiel

Rollen helfen dem LLM, den richtigen Kontext und die passende Perspektive einzunehmen.

Du bist ein erfahrener Software-Architekt mit 15 Jahren Erfahrung in Enterprise-Anwendungen.

Entwerfe die Architektur für ein E-Commerce-System mit folgenden Anforderungen:

- Microservices-Architektur
- Skalierbarkeit für 1 Million Nutzer
- 99.9% Verfügbarkeit

Template-basiertes Prompting

Template-basiertes Prompting nutzt wiederverwendbare Strukturen für wiederkehrende Aufgaben.

- **Definition:** Wiederverwendbare Prompt-Strukturen
- **Anwendung:** Für wiederkehrende Aufgaben
- **Vorteile:** Konsistenz, Effizienz, Wartbarkeit
- **Struktur:** Variablen-Platzhalter für dynamische Inhalte

Template: Beispiel

Templates ermöglichen konsistente Prompts mit variablen Inhalten.

Du bist ein Code-Review-Experte. Analysiere folgenden Code:

****Code:****

{code_snippet}

****Anforderungen:****

- Prüfe auf Sicherheitslücken
- Bewerte Code-Qualität
- Gib konkrete Verbesserungsvorschläge

****Format:****

- Sicherheit: [Bewertung]
- Qualität: [Bewertung]
- Verbesserungen: [Liste]

Common Pitfalls und wie man sie vermeidet

Pitfall 1: Zu vage Formulierungen

Vage Formulierungen führen zu unklaren oder unerwünschten Ergebnissen.

Problem:

Mache etwas Nettes.

Lösung:

Erstelle eine freundliche Begrüßungs-E-Mail für neue Kunden, die sich für unseren Newsletter angemeldet haben. Die E-Mail soll willkommen heißen und die wichtigsten Vorteile des Newsletters erwähnen.

Pitfall 2: Fehlender Kontext

Ohne Kontext kann das LLM nicht wissen, welche Daten oder welches Format gemeint ist.

Problem:

Analysiere die Daten.

Lösung:

Analysiere die Verkaufsdaten aus der Datei 'sales_2024.csv'. Erstelle eine Zusammenfassung mit:

- Gesamtumsatz pro Monat
- Top 5 Produkte nach Umsatz
- Trend-Analyse für das letzte Quartal

Pitfall 3: Zu viele Anforderungen auf einmal

Zu viele Anforderungen überfordern das LLM und führen zu unvollständigen Ergebnissen.

Problem:

Erstelle eine vollständige Web-Anwendung mit Authentifizierung, Datenbank, API, Frontend, Tests, Dokumentation und Deployment-Skripten.

Lösung:

Erstelle zunächst die Grundstruktur einer Web-Anwendung:

1. Backend-API mit Express.js
2. Einfache Authentifizierung mit JWT
3. REST-Endpoints für CRUD-Operationen

Wir erweitern die Anwendung in weiteren Schritten.

Pitfall 4: Iterationen

Die erste Prompt-Version ist selten optimal und sollte iterativ verbessert werden.

- **Testing:** Verschiedene Varianten testen
- **Feedback:** Ergebnisse analysieren und anpassen
- **Dokumentation:** Erfolgreiche Prompts dokumentieren

Pitfall 5: Zu viele und widersprüchliche Vorgaben

Widersprüchliche Anforderungen verwirren das LLM und führen zu inkonsistenten Ergebnissen.

Problem:

Erstelle einen kurzen, detaillierten Text, der präzise ist, aber auch kreativ und informativ, sowohl für Anfänger als auch für Experten verständlich, formell aber auch locker, auf Deutsch und Englisch gleichzeitig.

Lösung:

Erstelle einen informativen Text über Machine Learning (ca. 200 Wörter) für Anfänger.

Anforderungen:

- Klare, einfache Sprache
- Konkrete Beispiele
- Format: Deutsch, formeller Ton

Prompt-Optimierung und Iteration

Der Optimierungsprozess

Prompt-Optimierung ist ein iterativer Prozess zur kontinuierlichen Verbesserung.

1. **Initialer Prompt:** Erste Version erstellen
2. **Testing:** Mit verschiedenen Eingaben testen
3. **Analyse:** Ergebnisse evaluieren
4. **Anpassung:** Prompt verfeinern
5. **Wiederholung:** Iterieren bis zufriedenstellend

Beispiel: Iterative Verbesserung

Iterative Verbesserung zeigt, wie Prompts schrittweise optimiert werden können.

Version 1:

Schreibe Code.

Version 2:

Schreibe Python-Code für eine Funktion.

Version 3:

Schreibe eine Python-Funktion, die eine Liste von Zahlen sortiert und das Maximum zurückgibt.

Version 4:

Erstelle eine Python-Funktion `find_max` mit folgenden Spezifikationen:

- **Eingabe:** Liste von Zahlen (int oder float)
- **Ausgabe:** Maximum der Liste (int oder float)

Praxisbeispiele

Beispiel 1: REST-Server mit Node.js

Erstellen Sie einen REST-Server mit Node.js und Express für ein Todo-Management-System.

Aufgabe:

Erstelle einen REST-Server mit Node.js und Express für ein Todo-Management-System.

Anforderungen:

- CRUD-Operationen (Create, Read, Update, Delete)
- JSON als Datenformat
- Fehlerbehandlung
- Strukturierter Code

Beispiel 1: Prompt-Struktur

Ein strukturierter Prompt mit klaren Anforderungen führt zu besseren Ergebnissen.

Du bist ein erfahrener Node.js-Entwickler. Erstelle einen REST-Server mit Express.js für ein Todo-Management-System.

****Anforderungen:****

- Framework: Express.js
- Datenformat: JSON
- Endpoints:
 - GET /todos - Liste aller Todos
 - POST /todos - Neues Todo erstellen
 - GET /todos/:id - Einzelnes Todo abrufen
 - PUT /todos/:id - Todo aktualisieren
 - DELETE /todos/:id - Todo löschen
- Fehlerbehandlung: 404 für nicht gefundene Todos
- Code-Struktur: Modulare Organisation

****Ausgabe:****

- Vollständiger Code mit Kommentaren
- package.json mit Dependencies
- README mit Anleitung

- <https://expressjs.com/>

Beispiel 2: Web-Anwendung mit React.js

Erstellen Sie eine React.js Web-Anwendung für ein einfaches Todo-Management.

Aufgabe:

Erstelle eine React.js Web-Anwendung für ein einfaches Todo-Management.

Anforderungen:

- Komponenten-basierte Architektur
- State Management
- Benutzerfreundliche UI
- Responsive Design

Beispiel 2: Prompt-Struktur

Ein detaillierter Prompt mit spezifischen Anforderungen führt zu besseren Ergebnissen.

Du bist ein Frontend-Entwickler mit Expertise in React.js. Erstelle eine Todo-Management-Web-Anwendung.

****Anforderungen:****

- Framework: React.js (mit Hooks)
- Features:
 - Todo-Liste anzeigen
 - Neue Todos hinzufügen
 - Todos als erledigt markieren
 - Todos löschen
 - Filter: Alle / Aktiv / Erledigt
- UI: Modern, benutzerfreundlich
- Styling: CSS oder Tailwind CSS
- State Management: useState/useReducer

****Ausgabe:****

- Vollständige Komponenten-Struktur
- Funktionaler Code mit Kommentaren
- Styling-Implementierung
- README mit Setup-Anleitung

- <https://react.dev/>

Zusammenfassung

Was haben wir heute gelernt?

Diese Vorlesung hat die Grundlagen des Prompt-Engineerings vermittelt.

- **Prompt-Engineering:** Systematische Gestaltung von LLM-Eingaben
- **Grundprinzipien:** Klarheit, Kontext, Struktur
- **Techniken:** Zero-shot, Few-shot, Chain-of-Thought, Role-based, Template-basiert
- **Pitfalls:** Häufige Fehler und wie man sie vermeidet
- **Optimierung:** Iterativer Verbesserungsprozess

Wichtige Erkenntnisse

Prompt-Engineering erfordert systematisches Vorgehen und kontinuierliche Verbesserung.

- **Keine Universallösung:** Prompts müssen an Aufgabe und Modell angepasst werden
- **Iteration ist essentiell:** Erste Version selten optimal
- **Struktur hilft:** Formatierung und Organisation verbessern Ergebnisse
- **Kontext ist wichtig:** Relevante Informationen bereitstellen
- **Testing notwendig:** Verschiedene Varianten ausprobieren

Nächste Schritte

Die nächste Vorlesung behandelt Retrieval-Augmented Generation (RAG).

- **Vorlesung 7:** Retrieval-Augmented Generation (RAG)
- **Praxis:** Eigene Prompts entwickeln und optimieren
- **Experimentieren:** Verschiedene Techniken ausprobieren
- **Fragen:** Nutzen Sie die Q&A-Zeit

Fragen und Diskussion

